

The Current Dynesty-Style `rwalk` Implementation in NISMO

Implementation specification for `nismo` version 0.1.0.dev3

August 9, 2026

Abstract

This document specifies the standard `proposal_scheme="rwalk"` kernel currently implemented by NISMO. It follows the actual Python execution path, including configuration resolution, construction and caching of the bounding ellipsoid, uniform-ball proposals, the fixed- q_0 Metropolis correction, scale adaptation, tie handling, resource limits, and recorded diagnostics. The controller and ellipsoid mechanics are adapted from Dynesty 3.1.0, while the target density and acceptance rule are specific to NISMO.

Contents

1	Scope and implementation locations	3
2	The constrained density targeted by the walk	3
3	Public configuration and exact resolution	3
4	Starting state and constraint ordering	4
5	Dynesty-style single bounding ellipsoid	4
5.1	Input live set	4
5.2	Condition-number repair	5
5.3	Expansion to contain every live point	5
5.4	Cache lifetime	6
6	One ellipsoidal-ball proposal	6
7	Evaluation and Metropolis decision	6
8	Exact complete-walk algorithm	7
9	Scale adaptation	7
10	Resource limits and failure semantics	8

NISMO standard <code>rwalk</code>	2
11 Recorded diagnostics	8
12 Why the kernel preserves the desired density	9
13 Usage	9
14 Attribution	10

1 Scope and implementation locations

This document concerns only the standard "`rwalk`" scheme. The separate "`en-rwalk`" differential-evolution ensemble kernel is not changed by this implementation and retains its own covariance shrinkage and jitter settings.

The implementation is divided among three modules:

- `src/nismo/config.py`: public `RWalkSettings`;
- `src/nismo/mcmc.py`: controller, ellipsoid, proposal, Metropolis transition, tuning, and method citation;
- `src/nismo/sampler.py`: run-level controller lifetime, dispatch, public citation property, live-set replacement, and histories.

No Dynesty package is imported at runtime. The adapted source is distributed under Dynesty's MIT terms in the repository's NOTICE file.

2 The constrained density targeted by the walk

Let $L(\theta)$ be the likelihood, $\pi(\theta)$ the normalized prior, and $q_0(\theta)$ the fixed normalized importance density supplied as `importance_morph`. NISMO defines

$$\log \Psi_0(\theta) = \log L(\theta) + \log \pi(\theta) - \log q_0(\theta). \quad (1)$$

At one nested iteration, the discarded live point defines the threshold λ . The replacement kernel must preserve

$$p_\lambda(\theta) \propto q_0(\theta) \mathbf{1}[\log \Psi_0(\theta) > \lambda]. \quad (2)$$

This differs from Dynesty's usual unit-cube construction. Dynesty obtains the prior density through a prior transform, whereas NISMO walks directly in physical parameter space. Consequently, merely passing the constraint is not enough: NISMO must also apply a Metropolis ratio involving q_0 .

3 Public configuration and exact resolution

The user passes an immutable `RWalkSettings` object:

```
1 RWalkSettings(
2     walks=None,
3     facc=0.5,
4     ncdim=None,
5 )
```

For model dimension D , the run-level `RWalkSampler` resolves the values as follows.

Field	Resolved value	Validation and meaning
<code>walks</code>	$D + 20$ when omitted	An explicit value must be a positive integer and is raised to a minimum of two. One successful replacement attempts exactly this many transitions.
<code>facc</code>	$\min(1, \max(1/W, f_{\text{input}}))$	The input must be finite. Here W is the resolved number of walks.
<code>ncdim</code>	D when omitted	An explicit value must equal D . Partial-dimensional refreshes cannot be implemented correctly for an arbitrary correlated importance Morph.
<code>initial scale</code>	$s_0 = 1$	Scale is controller state and is not a public setting.

The settings `n_steps`, `scale`, `covariance_shrinkage`, and `covariance_jitter` are not accepted by standard `rwalk`. Supplying them causes normal Python constructor errors.

Each call to `NISMOSampler.run()` constructs a fresh controller, so every run begins with scale one, zero acceptance history, and no cached ellipsoid.

4 Starting state and constraint ordering

The current worst live point supplies λ but is never a chain state. The implementation first identifies all surviving live indices that lie strictly above the active constraint, removes the worst index explicitly, and selects one eligible survivor uniformly. If none exists, the replacement fails with `insufficient_eligible_survivors`.

Under `tie_policy="strict"`, a proposal passes when

$$\log \Psi_0(\theta') > \lambda. \quad (3)$$

Under `tie_policy="randomized_plateau"`, every live state also contains an auxiliary $t \in [0, 1)$. The discarded point defines (λ, t_λ) , every proposal draws a fresh t' , and passage is lexicographic:

$$\log \Psi_0(\theta') > \lambda \quad \text{or} \quad [\log \Psi_0(\theta') = \lambda \text{ and } t' > t_\lambda]. \quad (4)$$

A rejected transition retains both the current parameter vector and its current auxiliary tie value.

5 Dynesty-style single bounding ellipsoid

5.1 Input live set

When no cached axes are available, or when the cache interval has elapsed, the implementation constructs a single ellipsoid from the complete current live matrix

$$\Theta = (\theta_1, \dots, \theta_N)^\top \in \mathbb{R}^{N \times D}. \quad (5)$$

This includes the point defining the threshold. Inclusion affects only the proposal geometry; that point remains forbidden as a chain start. At least two finite live points and one dimension are required.

Let

$$c = \frac{1}{N} \sum_{i=1}^N \theta_i, \quad C = \text{cov}(\Theta), \quad \delta_i = \theta_i - c, \quad (6)$$

where NumPy's sample covariance is used. In one dimension the scalar result is promoted to a 1×1 matrix.

5.2 Condition-number repair

The covariance repair attempts at most 100 eigendecompositions. Let $\ell_{\max}$ and $\ell_{\min}$ be the largest and smallest eigenvalues. The target maximum condition number is 10^{12} .

If the eigenvalues are finite and $\ell_{\max} > 0$, but

$$\ell_{\min} < \frac{\ell_{\max}}{10^{12}}, \quad (7)$$

the eigenvalues are floored at

$$\ell_j^{\text{fix}} = \max\left(\ell_j, \frac{10\ell_{\max}}{10^{12}}\right), \quad (8)$$

and the covariance is reconstructed. If the spectrum is nonfinite, has no positive eigenvalue, or eigendecomposition fails, the code blends toward the identity:

$$C \leftarrow (1 - a_k)C + a_k I, \quad (9)$$

where a_k grows geometrically from 10^{-10} to one over the 100 attempts. If no safe result is found, the code emits a runtime warning and uses a unit sphere, $C = P = A = I$.

For a successful spectrum $C = V \text{diag}(\ell) V^T$, the precision and proposal axes are

$$P = V \text{diag}(\ell^{-1}) V^T, \quad A = V \text{diag}(\sqrt{\ell}). \quad (10)$$

5.3 Expansion to contain every live point

The maximum squared ellipsoidal distance is

$$d_{\max} = \max_i \delta_i^T P \delta_i. \quad (11)$$

If $d_{\max} > 0.999$, the implementation sets

$$m = \frac{d_{\max}}{0.999}, \quad C \leftarrow mC, \quad P \leftarrow P/m, \quad A \leftarrow \sqrt{m} A. \quad (12)$$

The repair-and-expansion process receives at most two passes. Failure to obtain an ellipsoid with every live point strictly inside unit ellipsoidal distance raises `NumericalInvariantError`.

5.4 Cache lifetime

The axes are cached across replacements. With W walks and N live points, the refresh threshold is

$$WN \quad \text{completed random-walk calls.} \quad (13)$$

Each successful replacement adds exactly W to the cache counter. Thus, in the serial implementation, the bound is normally rebuilt before every N -th subsequent replacement. Interrupted or preflight-rejected chains do not advance this controller counter because the run stops without completing the replacement.

6 One ellipsoidal-ball proposal

For each transition, draw

$$z \sim \mathcal{N}(0, I_D), qquad u \sim \text{Uniform}(0, 1), qquad r = u^{1/D} \frac{\|z\|_2}{\|z\|_2}. \quad (14)$$

Then r is uniform in the unit D -ball and the proposal is

$$\theta' = \theta + sAr, \quad (15)$$

where A is the cached ellipsoid axis matrix and s is the scale frozen at the start of the complete walk. Equation (15) is symmetric about the current state. NISMO does not clip, reflect, wrap, or redraw a physical-space proposal. Prior support is handled by ordinary evaluation and rejection.

7 Evaluation and Metropolis decision

Every proposed parameter vector is evaluated exactly once by `BatchEvaluator`, producing cached values

$$(\theta', \log L', \log \pi', \log q'_0, \log \Psi'_0). \quad (16)$$

If the proposal fails the active pseudo-likelihood/tie constraint, it is rejected without an MH random draw. If it passes, the symmetric-proposal log acceptance probability is

$$\log \alpha = \min(0, \log q_0(\theta') - \log q_0(\theta)). \quad (17)$$

The implementation accepts when

$$\log U < \log \alpha, \quad U \sim \text{Uniform}(0, 1). \quad (18)$$

An accepted transition replaces every cached field, including the tie value. A rejected transition changes none of them. A proposed $\log q_0 = -\infty$ has zero acceptance probability; other nonfinite proposal densities raise a numerical invariant error. The eligible starting state must have finite $\log q_0$.

8 Exact complete-walk algorithm

For one nested replacement, the implemented sequence is:

1. Validate live-array shapes and preflight the resource limits for all W proposals.
2. Identify eligible survivors. Fail if there are none.
3. Obtain the cached or newly rebuilt ellipsoid axes from the complete live set.
4. Select one eligible survivor uniformly and copy its complete cached state.
5. Freeze the controller’s current scale for this walk.
6. Repeat exactly W times:
 - (a) check the wall-time deadline;
 - (b) draw a uniform-ball displacement and form θ' ;
 - (c) evaluate θ' once and draw its tie value;
 - (d) test the active constraint;
 - (e) if valid, apply Equation (17);
 - (f) on acceptance, replace the complete current state.
7. Return the actual final state, even if no transition was accepted.
8. Record W completed calls and tune the scale for the next nested replacement.

The following compact pseudocode emphasizes the ordering:

```

1 axes = sampler.axes_for(live_theta)
2 current = copy(uniform_choice(eligible_survivors))
3 scale_used = sampler.scale
4
5 for _ in range(sampler.walks):
6     r = uniform_unit_ball(ndim)
7     proposed = evaluate(current.theta + scale_used * axes @ r)
8     proposed.tie = rng.random()
9
10    if not passes_constraint(proposed):
11        continue
12    n_valid += 1
13
14    log_alpha = min(0.0, proposed.log_q0 - current.log_q0)
15    if log(rng.random()) < log_alpha:
16        current = proposed
17        n_accepted += 1
18
19 sampler.record_completed_walk(
20     accept=n_accepted,
21     scale=scale_used,
22 )
23 return current

```

9 Scale adaptation

Let A_k and R_k denote accepted and rejected transitions accumulated in the controller. In the current serial NISMO path, tuning updates after every completed replacement, so normally

$$A_k + R_k = W, \quad f_k = \frac{A_k}{A_k + R_k}. \quad (19)$$

Here “accepted” means final MH acceptance. Constraint failures and valid but MH-rejected proposals both contribute to R_k . With target fraction $f_0 = \text{facc}$, the Dynesty recursion is

$$s_{k+1} = s_k \exp \left(\frac{f_k - f_0}{D f_0} \right). \quad (20)$$

Acceptance above target expands the next walk; acceptance below target shrinks it. After Equation (20), both accumulated history counters reset to zero. The optional `update=False` controller path only accumulates history and is retained from the Dynesty-style interface, but the current serial NISMO dispatch updates immediately after a completed walk.

10 Resource limits and failure semantics

Before constructing a bound or starting a chain, NISMO rejects the complete replacement without any new likelihood calls when:

- $W > \text{max_proposals_per_replacement}$;
- the run-wide likelihood budget cannot accommodate all W calls;
- the wall-time deadline has already elapsed.

The wall-time deadline is also checked before every scalar transition. If it expires inside a walk, the replacement returns no draw, the worst live point is left untouched, and the shortened chain is never used. A successful replacement always has

$$n_{\text{proposed}} = n_{\text{likelihood calls}} = n_{\text{completed}} = W. \quad (21)$$

This equality applies to calls made by the replacement itself; initial live point evaluation is additional run-wide work.

11 Recorded diagnostics

For each completed nested replacement, `RunHistory` records:

Field	Standard <code>rwalk</code> meaning
<code>constraint_pass_fraction</code>	Number passing the active pseudo-likelihood/tie constraint divided by W .
<code>mh_acceptance_fraction</code>	Number accepted by the complete constraint-plus-MH transition divided by W .
<code>mcmc_accepted</code>	Count of accepted transitions in the completed walk.
<code>mcmc_moved</code>	One if at least one transition was accepted, otherwise zero.
<code>mcmc_completed</code>	Exactly W .
<code>acceptance_fraction</code>	Cumulative nested replacements divided by all proposals; this retains the sampler-wide efficiency meaning and is not the MH rate.

The adapted scale and ellipsoid axes are controller state and are not added as public `RunHistory` arrays.

12 Why the kernel preserves the desired density

During one complete walk, A and s are frozen. The uniform-ball increment is centered and symmetric, hence

$$Q(\theta' | \theta) = Q(\theta | \theta'). \quad (22)$$

Inside the permitted constraint, the target-density ratio from Equation (2) is exactly

$$\frac{p_\lambda(\theta')}{p_\lambda(\theta)} = \frac{q_0(\theta')}{q_0(\theta)}, \quad (23)$$

which gives Equation (17). Proposals outside the constraint have target density zero and are rejected. Bound and scale adaptation occurs only between complete walks, so each individual transition kernel retains the required symmetry.

Invariance does not imply independence. The returned point remains correlated with its selected live survivor, especially when W is too small or the acceptance rate is poor. Users should compare repeated runs, monitor the separate constraint and MH fractions, and test sensitivity to `walks` and `n_live`. Conditioning the output on movement, forcing an acceptance, or choosing the highest-likelihood visited state would change the target and is not implemented.

13 Usage

Explicit control:

```

1 from nismo import NISMOSampler, RWalkSettings
2
3 sampler = NISMOSampler(
4     model=model,
5     importance_morph=importance_morph,
6     proposal_scheme="rwalk",
7     rwalk_settings=RWalkSettings(
8         walks=50,
9         facc=0.5,
10    ),
11    n_live=200,
12    rng=42,
13 )
14
15 result = sampler.run(
16     dlogz=0.1,
17     max_proposals_per_replacement=100,
18 )

```

Dynesty-style top-level defaults:

```

1 sampler = NISMOSampler(
2     model=model,
3     importance_morph=importance_morph,
4     proposal_scheme="rwalk",
5     rwalk_settings=RWalkSettings(),
6     n_live=200,
7     rng=42,
8 )

```

For this second example, $W = D + 20$, $f_0 = 0.5$, and $s_0 = 1$. The hard proposal limit supplied to `run()` must be at least W .

Useful inspection:

```
1 print(sampler.citations)
2 print(result.history.constraint_pass_fraction)
3 print(result.history.mh_acceptance_fraction)
4 print(result.history.mcmc_moved)
```

For standard `rwalk`, `sampler.citations` returns only the Skilling (2006) method entry, as specified by the current public behavior.

14 Attribution

The stateful controller, uniform-unit-ball draw, covariance repair, and single-ellipsoid construction are adapted from the Dynesty 3.1.0 source:

<https://github.com/joshspeagle/dynesty/tree/v3.1.0>

Dynesty’s copyright and MIT permission notice are reproduced in the root `NOTICE` file and included in source and wheel distributions. The runtime method citation remains the reference below.

References

- [1] J. Skilling, “Nested sampling for general Bayesian computation,” *Bayesian Analysis*, vol. 1, no. 4, pp. 833–859, 2006. <https://projecteuclid.org/euclid.ba/1340370944>